The Journal of Open Source Software

Discovering the SUPER in computing - dagster-slurm for reproducible research on HPC

Hernan Picatto [2], **Maximilian Heß** [2], and **Georg Heiler** [1,2]

1 Complexity Science Hub Vienna (CSH) **2** Austrian Supply Chain Intelligence Institute (ASCII)

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Christoph Junghans

Reviewers:

- @MoraruMaxim

Submitted: 31 October 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

In partnership with

AMERICAN ASTRONOMICAL SOCIETY

This article and software are linked with research article DOI 10.3847/xxxxx, published in the Journal of Open Source Software.

DRAFT

Summary

Dagster is a modern data orchestrator that emphasises reproducibility, observability, and a strong developer experience (Elementl, 2024b). In parallel, most high-performance computing (HPC) centres continue to rely on Slurm for batch scheduling and resource governance (Yoo et al., 2003). The two ecosystems rarely meet in practice: Dagster projects often target cloud or single-node deployments, while Slurm users maintain bespoke submission scripts with limited reuse or visibility. This paper introduces **dagster-slurm**, an open-source integration that allows the same Dagster assets to run unchanged across laptops, CI pipelines, containerised Slurm clusters, and Tier-0 supercomputers. The project packages dependencies with Pixi (prefix.dev, 2024), submits workloads through Slurm using Dagster Pipes (Elementl, 2024a), and streams logs plus scheduler metrics back to the Dagster UI.

The key contribution is a unified compute resource (`ComputeResource`) that hides SSH transport (including password-only jump hosts and OTP prompts), dependency packaging, and queue configuration while still respecting Slurm's scheduling semantics. Today the project ships with two production-ready execution modes—`local` for laptop/CI development and `slurm` for one-job-per-asset submissions. Experimental support for session-based reuse and heterogeneous jobs is work in progress. Launchers for Bash, Ray, and Spark (work in progress) ship out of the box and can be extended.

Statement of Need

Research software engineers (RSE) and data scientists increasingly face cross-environment workflows on it's 22 edition the Research Software Engineering (RSE) International Survey (Hettrick et al., 2022) every country marked HPC as important skill for RSE. Reproducibility challenges in HPC environments are well-documented, (Antunes & Hill, 2024) provided comprehensive coverage of issues unique to HPC in their survey. (Courtès, 2022) further examine the tension between reproducibility and performance, demonstrating that these goals need not be mutually exclusive.

The 2024 Community Workshop on Practical Reproducibility in HPC produced a comprehensive report highlighting cost-effective reproducibility challenges and the need for tools that bridge development and production environments (Keahey et al., 2025). Existing solutions either ignore the orchestrator (hand-written Slurm scripts) or bypass the scheduler entirely (running ad-hoc services). This leads to duplicated logic, fragile deployments, and a loss of telemetry. **dagster-slurm** was created to:

- Preserve Dagster's asset-based design, lineage tracking, and alerting for workloads that ultimately run on Slurm-managed hardware.
- Remove the need to rewrite orchestration glue when moving from development to production supercomputers.

- Provide a batteries-included path for packaging Python environments reproducibly (Pixi https://pixi.sh/latest/ + pixi-pack, https://github.com/Quantco/pixi-pack) and deploying them in air-gapped environments.
- Offer a clear path to advanced HPC patterns (session reuse for long-lived clusters, heterogeneous Slurm jobs) while keeping the current stable surface area intentionally small; these features are being iterated on in the open.
- Encourage research software engineering best practices as outlined by (Eisty et al., 2025), covering planning, testing, documentation, and maintenance across the development lifecycle.

System Overview

The integration is composed of three layers:

1. **Resource definitions** – `ComputeResource`, `SlurmResource`, `SlurmSessionResource`, and `SSHConnectionResource` are Dagster `ConfigurableResource` objects. They encapsulate queue defaults, SSH authentication (including ControlMaster fallback, password-based jump hosts, and interactive OTP prompts), and execution modes.
2. **Launchers and Pipes clients** – Launchers (Bash, Ray) translate payloads into execution plans. The Slurm Pipes client handles environment packaging (on demand or via pre-deployed bundles), transfers scripts, triggers `sbatch` or session jobs, and streams logs/metrics back through Dagster Pipes (Elementl, 2024a). Spark and custom launchers are planned for near future release
3. **Operational helpers** – Environment deployment scripts, heterogeneous job managers, metrics collectors, and SSH pooling utilities target HPC constraints such as login-node sandboxes, session allocations, and queue observability.

This layered approach keeps Dagster's user code agnostic to the underlying transport while retaining the full control plane visibility of the orchestrator. As illustrated in Figure 1, the same scalable job can follow multiple paths: direct local execution for development, automated testing through CI/CD chains, or production deployment to HPC clusters via SSH-accessible edge nodes.

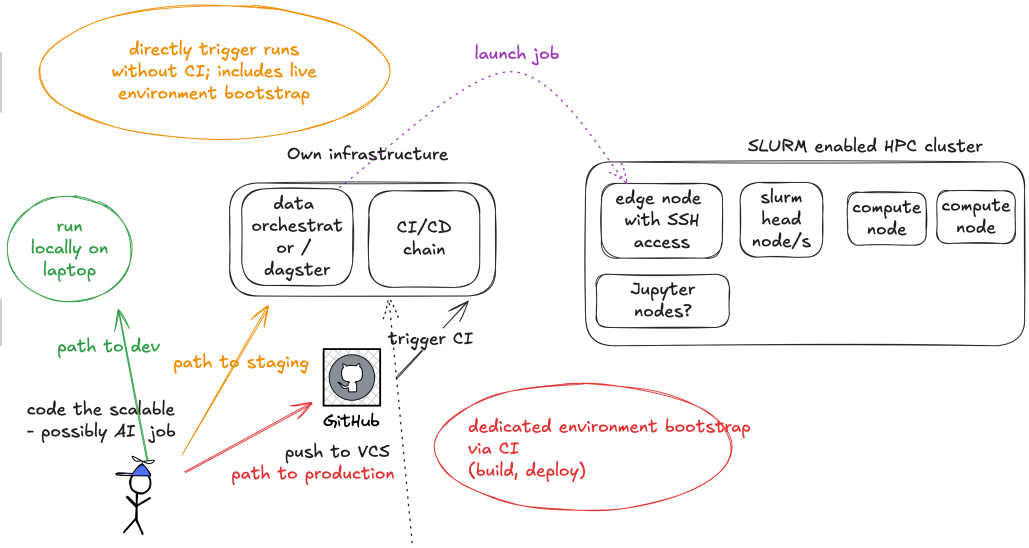

Figure 1: Architecture overview showing the progression from local development to production HPC deployment. The workflow enables researchers to code locally, test through CI/CD pipelines, and deploy to Slurm-enabled clusters with minimal changes.

Minimal usage example

```python
import dagster as dg
from dagster_slurm import (
    ComputeResource,
    ExecutionMode,
    RayLauncher,
    SlurmQueueConfig,
    SlurmResource,
    SSHConnectionResource,
)

ssh = SSHConnectionResource.from_env(prefix="SLURM_EDGE_NODE")
slurm = SlurmResource(
    ssh=ssh,
    queue=SlurmQueueConfig(partition="batch",
      time_limit="02:00:00", cpus=8, mem="32G"),
    remote_base=f"/home/{ssh.user}/dagster_runs",
)

compute = ComputeResource(
    mode=ExecutionMode.SLURM,
    slurm=slurm,
    default_launcher=RayLauncher(num_gpus_per_node=1),
)

@dg.asset(required_resource_keys={"compute"})
def train_model(context: dg.AssetExecutionContext):
    payload = dg.file_relative_path(__file__, "../workloads/train.py")
    completed = context.resources.compute.run(
        context=context,
        payload_path=payload,
        resource_requirements={"framework": "ray",
          "cpus": 32, "gpus": 1, "memory_gb": 120},
        extra_env={"EXPERIMENT": context.run.run_id},
    )
    yield from completed.get_results()
```

Local development simply swaps `ExecutionMode.SLURM` for `ExecutionMode.LOCAL`. The example project bundled with the repository demonstrates this workflow, complete with Dockerised Slurm nodes for integration testing. Session reuse and heterogeneous jobs remain under active development; early adopters can track progress in the repository milestones.

Evaluation

We validate the approach along three dimensions:

- **Reproducibility** – Integration tests run inside GitHub Actions using a containerised Slurm cluster. The pipeline provisions the environment with Pixi, deploys it once via `pixi run deploy-prod-docker`, and then runs Dagster assets through all four execution modes. This continuous integration approach aligns with emerging best practices for reproducible HPC workflows (Hayot-Sasson et al., 2025).
- **HPC readiness** – The project has been exercised on academic clusters such as VSC-5 (Austria) and Leonardo (Italy). SSH ControlMaster fallbacks, password-based jump hosts, `.bashrc` hygiene, queue/QoS/reservation overrides, and verification snippets (`squeue`,

DRAFT

`scontrol`) are documented for both sites.

- **Observability** – Slurm job IDs, CPU efficiency, memory, and node-hours are exposed as Dagster metadata entries, while Ray clusters stream their stdout/stderr back through Pipes. This enables conventional Dagster asset checks and alerting to operate unchanged.

Impact and Future Work

dagster-slurm lowers the barrier for research teams to adopt modern data orchestration on top of established HPC schedulers. By eliminating duplicated scripts and surfacing rich observability, the integration reduces operational toil and shortens iteration loops. Future work focuses on:

- Deepening heterogeneous job support (automatic fusion of dependent assets, richer allocation policies).
- Exploring pilot-job back ends (e.g., RADICAL-Pilot, QCG) for even finer-grained scheduling inside allocations, and non-interactive OTP integrations for environments with strict MFA policies.
- Extending language/runtime coverage via additional launchers (MPI, GPU-accelerated frameworks) and multi-language Pipes integrations.

Community contributions—issue reports, cluster-specific recipes, and new launchers—are actively encouraged at https://github.com/ascii-supply-networks/dagster-slurm.

Acknowledgements

We thank the operations teams at the Austrian Scientific Computing (ASC) (VSC-5) and CINECA's Leonardo supercomputer for early feedback, and the Dagster community for discussions around orchestrating HPC workloads. Funding and in-kind support were provided by the Complexity Science Hub Vienna and the Austrian Supply Chain Intelligence Institute. This work was completed in part at the EUROCC AI HACKATHON 2025, part of the Open Hackathons program. The authors would like to acknowledge OpenACC-Standard.org for their support.

References

Antunes, B., & Hill, D. R. C. (2024). Reproducibility, replicability and repeatability: A survey of reproducible research with a focus on high performance computing. *Computer Science Review*, *53*, 100655. https://doi.org/https://doi.org/10.1016/j.cosrev.2024.100655

Courtès, L. (2022). Reproducibility and performance: Why choose? *Computing in Science & Engineering*, *24*(3), 77–80. https://doi.org/https://doi.org/10.1109/MCSE.2022.3165626

Eisty, N. U., Bernholdt, D. E., Koufos, A., Luet, D. J., & Mundt, M. (2025). *Ten essential guidelines for building high-quality research software*. https://doi.org/https://doi.org/10.48550/arXiv.2507.16166

Elementl, Inc. (2024a). *Dagster pipes documentation*. https://docs.dagster.io/concepts/dagster-pipes/pipes-overview

Elementl, Inc. (2024b). *Dagster: Data orchestration platform*. https://dagster.io/

Hayot-Sasson, V., Hudson, N., Bauer, A., Gonthier, M., Foster, I., & Chard, K. (2025). *Addressing reproducibility challenges in HPC with continuous integration*. https://doi.org/https://doi.org/10.48550/arXiv.2508.21289

Hettrick, S., Bast, R., Crouch, S., Wyatt, C., Philippe, O., Botzki, A., Carver, J., Cosden, I., D'Andrea, F., Dasgupta, A., Godoy, W., Gonzalez-Beltran, A., Hamster, U., Henwood, S., Holmvall, P., Janosch, S., Lestang, T., May, N., Philips, J., … Zhang, Q. (2022).

DRAFT

International RSE survey 2022 (Version v0.9.3). Zenodo. https://doi.org/10.5281/zenodo.7015772

Keahey, K., Richardson, M., Tolosana Calasanz, R., Hunold, S., Lofstead, J., Malik, T., & Perez, C. (2025). *Report on challenges of practical reproducibility for systems and HPC computer science*. Zenodo. https://doi.org/10.5281/zenodo.15306610

prefix.dev. (2024). *Pixi: Reproducible package environments*. https://github.com/prefix-dev/pixi

Yoo, A. B., Jette, M. A., & Grondona, M. (2003). SLURM: Simple linux utility for resource management. In D. G. Feitelson, L. Rudolph, & U. Schwiegelshohn (Eds.), *Job scheduling strategies for parallel processing* (Vol. 2862, pp. 44–60). Springer. https://doi.org/https://doi.org/10.1007/10968987_3

DRAFT